

HOW THE USE OF DESIGN PATTERNS AFFECTS THE QUALITY OF SOFTWARE SYSTEMS: A PRELIMINARY INVESTIGATION

Carmine Gravino, Michele Risi

Software Quality and Measurement Laboratory (SQMLab)
Department of Computer Science
University of Salerno

Outline

Motivations and Previous Studies

Case study

Design

Results

Conclusions and future work

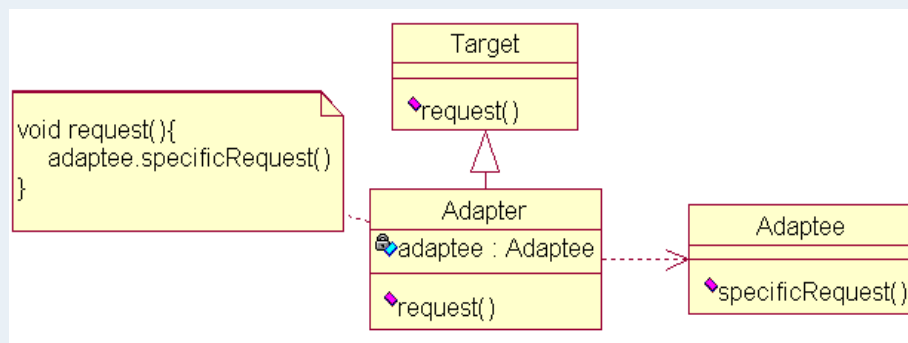
Design Patterns (DPs)

What is a design pattern?

4 essential elements: name, problem, solution, consequences

Some examples, intent, and so on ...

DP Instances (also known as design motifs)



Graphically documented

```
1: /*
2:  * @(#)CompositeFigure.java 5.1
3:  *
4:  * Composite Pattern
5:  * Component: Figure
6:  * Composite: CompositeFigure
7:  * Leaf: RectangleFigure
8:  *
9:  */
10:
11: package CH.ifa.draw.standard;
12:
13: ....
14:
15: public abstract class CompositeFigure
16:     extends AbstractFigure
17:     implements FigureChangeListener {
```

Textually documented

Design Patterns (DPs)

“DPs can improve the documentation and maintenance of existing systems by furnishing an explicit specification of class and object interactions and their underlying intent”

- helping the comprehension of the adopted solution
- highlighting wished properties of the design model
- guiding the restructuring of the system



Previous studies

Prechelt et al. [5] have performed a series of experiments to test whether pattern comment lines help during maintenance phase when they are well documented

Aversano et al. [7] have investigated the frequency and scope of modifications of design patterns

Scanniello *et al.* [2] have studied the importance of documenting design pattern instances and how it can affect the comprehensibility of source code

[5] L. Prechelt, B. Unger-Lamprecht, M. Philippsen, and W. Tichy, "Two controlled experiments assessing the usefulness of design pattern documentation in program maintenance" IEEE TSE, vol. 28(6), 2002.

[2] G. Scanniello, C. Gravino, M. Risi, G. Tortora, and G. Dodero, "Doc-umenting design-pattern instances: A family of experiments on source-code comprehensibility", ACM TOSEM 24(3) 2015.

[7] L. Aversano, G. Canfora, L. Cerulo, C. Del Grosso, and M. Di Penta, "An empirical study on the evolution of design patterns," in ESEC/FSE, 2007

Previous studies

Khomh et al. [9] assessed the impact that specific design pattern roles play on the quality of classes, analyzed in terms of Chidamber and Kemerer (CK) measures

- a significant increase in many metric values and in the frequency and numbers of changes when classes play two roles in design pattern instances

Our study can be considered a further assessment with respect to [9], exploiting more design patterns (20 vs. 6) and software systems (10 vs. 6)

Chidamber and Kemerer proposal

Metriche di Chidamber and Kemerer [A Metrics Suite for Object-Oriented Design, 1994]:

- weighted methods per class (WMC)
- depth of the inheritance tree (DIT)
- number of children (NOC)
- coupling between object classes (CBO)
- response for a class (RFC)
- lack of cohesion in methods (LCOM)

Weighted Method per Class (WMC)

WMC è la somma pesata dei metodi di una classe

Il peso di un metodo è dato da un fattore di complessità a scelta.

- Il fattore di complessità dei metodi può essere una metrica di complessità proposta per le funzioni (ad esempio la complessità di Mc Cabe).
- Al crescere di WMC aumenta la complessità della classe e diminuisce la speranza di poterla riusare.

Problema: come tener conto della complessità dei metodi ereditati?

Profondità dell'albero di ereditarietà di una classe (DIT)

Distanza massima di un nodo (una classe) dalla radice dell'albero rappresentante la struttura ereditaria

- Maggiore è la profondità della classe nella gerarchia, maggiore è il numero di metodi che essa può ereditare, rendendo più complesso predire il suo comportamento.
- Alberi di ereditarietà con elevata profondità possono aumentare la complessità del progetto (più classi e metodi sono coinvolti).
- Maggiore è la profondità di una classe in una gerarchia, maggiore è il riuso potenziale dei metodi ereditati.

Numero di figli (NOC)

Numero di sottoclassi, figlie di una super-classe.

Al crescere del NOC, cresce il livello di riuso (NOC è dunque un indicatore di Riuso) ...

ma tende ad 'evaporare' l'astrazione della Classe madre.

aumenta la possibilità che una Classe figlia non sia membro appropriato della madre.

Al crescere del NOC, cresce la quantità di test case necessari a testare ogni figlia.

Risposte per classe (RFC)

Insieme dei metodi che possono essere potenzialmente eseguiti in risposta ad un messaggio ricevuto da un oggetto della Classe

- Include l'insieme dei metodi della classe e di tutte le classi a cui essa invia messaggi

RFC è un indicatore del 'volume' di interazione fra classi.

A valori elevati di RFC:

- aumenta la complessità progettuale della classe;
- cresce lo sforzo per il testing.

Accoppiamento tra le classi (CBO)

Numero di collaborazioni di una classe, ovvero numero di altre classi cui essa è accoppiata.

- L'accoppiamento può avvenire a seguito di lettura/modifica attributi, chiamata metodi, creazione di oggetti;

Eccessivo accoppiamento è negativo per la modularità ed il riuso;

Maggiormente indipendente è una classe più facilmente è riusabile;

Per migliorare la modularità, l'accoppiamento va tenuto al minimo; esso influisce sull'impatto di modifiche in altri moduli;

Valori elevati del CBO complicano testing e modifiche.

Mancanza di coesione nei metodi (LCOM)

La coesione di un metodo esprime la proprietà di un metodo di accedere in maniera esclusiva ad attributi della classe.

- La Mancanza di Coesione deriva dalla presenza di più metodi che accedono ad attributi comuni.
- Se ogni attributo è acceduto da un solo metodo, allora $LCOM=0$ (massima coesione dei metodi).
- Altrimenti, $LCOM > 0$ ed aumenta quanto più i metodi vanno ad operare sugli stessi attributi, rendendo quindi difficile il controllo di tutte le interazioni tra i metodi dovuti all'uso degli attributi comuni.

Quindi se $LCOM$ è elevato, i metodi potrebbero essere molto accoppiati tra loro attraverso gli attributi, ed aumenta la complessità del design della classe.

Study design: Research questions

RQ_X: Are the classes participating in design pattern instances characterized by better values of the CK measure X with respect to the classes that are not involved in design pattern instances?

X can be

- WMC (Weighted Methods per Class)
- LCOM (Lack of Cohesion in Methods)
- AMS (AvgLineCode in methods)
- CLOC (CountLineComment)
- LOC (CountLineCode)
- CLOC/NL (CountLineComment/CountLine)

*For DIT (Depth of Inheritance) and
CBO (Coupling Between Object classes):
Analysis only for DP Classes*

Study design: Software systems

Software systems
widely used in the
literature

System	# pattern instances	# DP classes (and %)	# NoDP classes (and %)
HotDraw 5.1	68	119 (71)	49 (29)
JHotDraw 6.0	82	76 (23)	253 (77)
JUnit	14	25 (27)	68 (73)
JRefactory	41	198 (34)	392 (66)
QuickUML	18	54 (25)	162 (75)
PMD	14	52 (11)	440 (89)
Nutch	15	40 (13)	260 (87)
MapperXML	25	60 (24)	186 (76)
NetBeans	26	200 (4)	5347 (96)
Lexi	5	8 (6)	117 (94)

Design pattern instances from P-MARt and recovered by several tools: DPD, ePAD, RM, and DPF

CK measures obtained by using the tool Understand

Study design: Data analysis

Analysis of simple descriptive statistics

Mann-Whitney test to verify whether there is statistical significant difference among CK metrics values of DP and NoDP classes.

- Results statistically significant at $\alpha = 0.05$

Cliff's d non-parametric effect size measure to provide information about the magnitude of the difference between two distributions

- negligible ($d < 0.147$), small ($0.147 \leq d < 0.33$), medium ($0.33 \leq d \leq 0.474$), and large ($d > 0.474$)

Results

System	AMS	LOC	CLOC	LCOM	WMC	CLOC/NL
JHotDraw 5.1	=	DP	DP	DP	DP	=
JHotDraw 6.0	NoDP	DP	DP	DP	DP	=
JUnit	NoDP	DP	DP	DP	DP	DP
JRefactory	NoDP	NoDP	=	DP	DP	=
QuickUML	NoDP	DP	DP	DP	=	DP
Lexi	NoDP	DP	DP	NoDP	NoDP	DP
Nutch	DP	NoDP	=	NoDP	=	=
MapperXML	NoDP	NoDP	DP	DP	DP	DP
PMD	NoDP	DP	DP	DP	DP	=
NetBeans	NoDP	DP	DP	DP	DP	DP

"DP" for the cell (i,j) means that DP classes in the system *i* are characterized by a mean value for the measure *j* better than the one for the NoDP classes.

"=" for the cell (i,j) means that the mean values of the measure *j* calculated for DP and NoDP classes are comparable for the system *i*

Results

- DP classes are not characterized by better AMS values wrt NoDP classes
- *DP classes are characterized by a greater number of LOC*
- *DP classes are characterized by a greater number of CLOC*
- *DP classes characterized by a better result in terms of LCOM wrt to NoDP classes*
- *DP classes are more complex than NoDP classes (WMC)*
- *DP classes are characterized by a better result in terms of CLOC/NL wrt NoDP classes*

Results: Significant differences

System	AMS	LOC	CLOC	LCOM	WMC	CLOC/NL
JHotDraw 5.1	=	DP	DP	DP	DP	=
JHotDraw 6.0	NoDP	DP	DP	DP	DP	=
JUnit	NoDP	DP	DP	DP	DP	DP
JRefactory	NoDP	NoDP	=	DP	DP	=
QuickUML	NoDP	DP	DP	DP	=	DP
Lexi	NoDP	DP	DP	NoDP	NoDP	DP
Nutch	DP	NoDP	=	NoDP	=	=
MapperXML	NoDP	NoDP	DP	DP	DP	DP
PMD	NoDP	DP	DP	DP	DP	=
NetBeans	NoDP	DP	DP	DP	DP	DP

Medium effect size

Results for CBO e DIT

System	CBO	DIT
JHotDraw 5.1	Mean(DP classes)= 3.98	Mean(DP classes) = 3
JHotDraw 6.0	Mean(DP classes) = 10.87	Mean(DP classes) = 2
JUnit	Mean(DP classes) = 2.5	Mean(DP classes) = 2
JRefactory	Mean(DP classes) = 7.35	Mean(DP classes) = 2
QuickUML	Mean(DP classes) = 5	Mean(DP classes) = 2
Lexi	Mean(DP classes) = 2.5	Mean(DP classes) = 2
Nutch	Mean(DP classes) = 6.78	Mean(DP classes) = 1
MapperXML	Mean(DP classes) = 4.11	Mean(DP classes) = 3
PMD	Mean(DP classes) = 15.38	Mean(DP classes) = 1
NetBeans	Mean(DP classes) = 5.33	Mean(DP classes) = 2

For **CBO** (between 1 and 4): values in the range 1-2 are "low", values in the range 3-4 are "medium", values equal or higher than 5 are "high"

For **DIT** (between 0 and 4): values in the range 0-1 are "low", while higher values are "medium-high"

Summary and conclusions

Expected results:

- For the DP classes more lines of comments have been written by developers
- The methods of DP classes are characterized by a less number of lines of code with respect to the NoDP classes. This should improve the comprehensibility of the source code
- In general the developers wrote more lines of code for a class in the set of DP classes with respect to NoDP classes

Summary and conclusions

Unexpected results:

- The level of cohesion seems to be lower for DP classes
- WMC values has highlighted that the DP classes are also more complex
- DP classes are also characterized by a high level of coupling. So, complex code.

We are in the case: we structure our code in a more complex way but this choice allow us to re-use available and tested solutions that can also improve comprehensibility of the code if the pattern instances are well documented

- further investigations in the future

Future work

Further studies with other and different software systems

Analysis of the relationships among:

- CK measure values
- Classes involved in Design patterns
- Classes involved in bad smells

Study design: DP data

	JHotDraw 5.1			JHotDraw 6.0			JUnit			JRefractory			QuickUML		
Design pattern	#	DP	NoDP	#	DP	NoDP	#	DP	NoDP	#	DP	NoDP	#	DP	NoDP
Factory method	6	38	130	10	7	322				8	8	582			
Decorator	1	24	144				1	5	88						
Singleton	2	2	166				2	2	91	2	2	588	1	1	215
Composite	1	23	145				1	3	90				2	16	200
Adapter	1	30	138							17	32	558			
Prototype	3	23	145	3	3	326							2	2	214
Command	13	22	146	19	21	308							6	6	210
Observer	8	45	123	11	19	310	3	10	83	3	7	583	3	20	196
State	15	21	147	15	13	316	1	3	90	2	8	582	1	2	214
Strategy	15	34	134	22	21	308	3	7	86				2	3	213
Template method	2	23	145	1	1	328	2	2	91	5	5	585	1	1	215
Null object	1	1	167												
Visitor				1	1	328				2	136	454			
Iterator							1	3	90						
Builder										2	7	583	1	13	203
Abstract factory													1	9	2017
	PMD			Nutch			MapperXML			NetBeans			Lexi		
Design pattern	#	DP	NoDP	#	DP	NoDP	#	DP	NoDP	#	DP	NoDP	#	DP	NoDP
Factory method	3	15	477				1	26	220						
Decorator															
Singleton				1	1	299	3	3	243				2	2	123
Composite	2	6	486				1	25	221						
Adapter	1	3	489	2	4	296	2	4	242	8	65	5282			
Prototype															
Command				2	6	294				1	32	5315			
Observer	2	11	481				11	32	214				2	3	122
State															
Strategy				2	8	292	1	3	243						
Template method	1	1	491	3	14	286	4	35	211						
Null object															
Iterator	1	2	490	1	6	294				5	10	5337			
Visitor	1	5	487												
Proxy	1	3	489												
Builder	2	13	479										1	5	120
Memento				2	10	290									
Bridge				2	22	278									
Facade							1	8	238						
Abstract factory							1	26	220	12	111	5236			